

Introduction to Testing Methodologies

Strategies, approaches, and methods used to verify that software behaves as specified.

— Software Testing Module 1

Today's Agenda

Four questions we'll answer by the end of this session

01



What is software testing, really?

02



What do we mean by a 'testing methodology'?

03



Why does it matter for real products?

04



What are the major categories & levels?

What Is Software Testing?



The process of evaluating a software product to find whether it meets specified requirements and to identify defects.

Think of it as quality control for code — the same instinct that makes you double-check a bridge before opening it to traffic.



Confirms correctness

The product works as per the given specifications — it does what it was built to do.



Confirms safety

It has no harmful side effects even when used outside the expected design parameters.

What Are Testing Methodologies?

Definition: Testing strategies, approaches, or methods used to test a specific product to ensure its usability.

In short — a methodology is your game plan for testing, not a single test itself.



Manual approaches

A human tester walks through the software step by step, like a user would.



Automated approaches

Scripts and tools execute test cases repeatedly, quickly, and consistently.



Structured strategies

Documented plans that decide what to test, in what order, and with what coverage.

Why Testing Methodologies Matter

Two questions every methodology is really trying to answer:

Verification

“Are we building the product right?”

Checks that the software conforms to its design and specifications at every stage of development.

Validation

“Are we building the right product?”

Checks that the finished software actually satisfies the user's real-world needs.

What happens when this discipline is skipped:

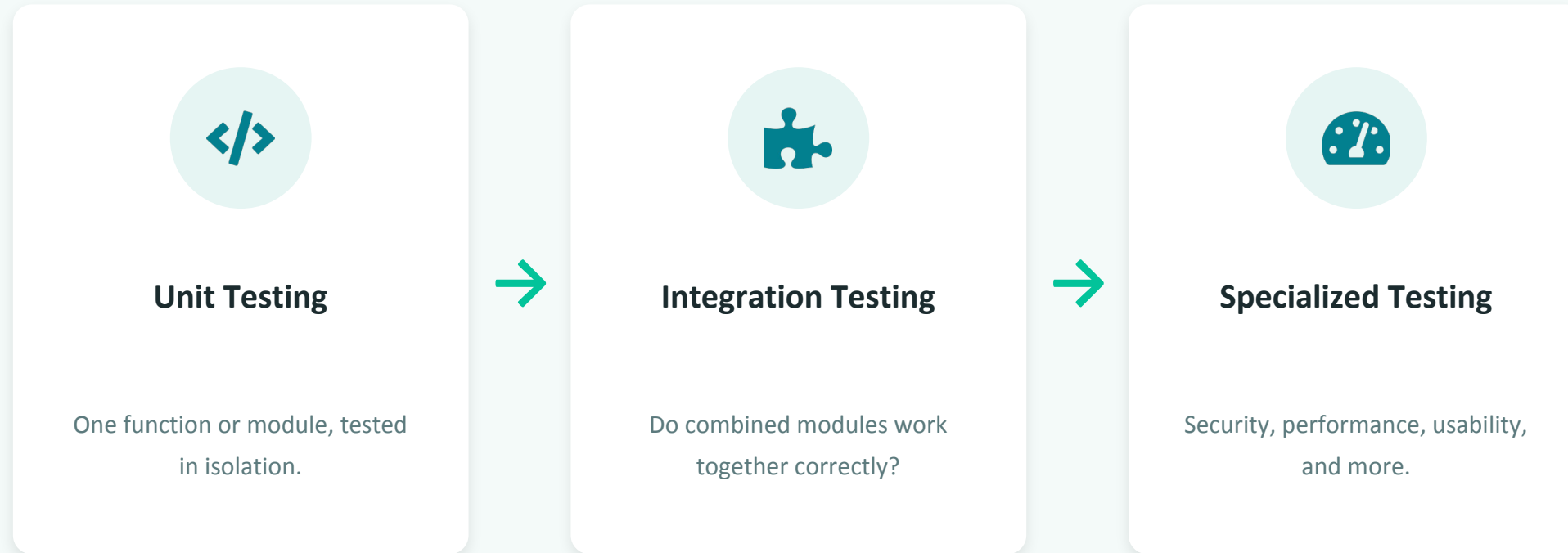
Bugs reach real users

Security gaps go unnoticed

Performance fails under load

The Testing Spectrum

From a single function to the entire system — testing scales up in scope.



A methodology decides how much of this spectrum you cover, and in what order, for a given release.

Levels of Testing

Each level answers a different question about the software.



Unit

Does this one piece of code work correctly on its own?

e.g. testing a single 'calculateTotal()' function



Integration

Do connected modules exchange data correctly?

e.g. cart module talking to the payment module



System

Does the complete, assembled application work end to end?

e.g. running the whole food-delivery app



Acceptance

Does it satisfy the client or user's actual needs?

e.g. client sign-off before go-live

How Methodologies Are Classified

Testing spans everything from unit checks to specialized security or performance testing.

By execution



Manual

A person runs the tests



Automated

Tools/scripts run the tests

By code visibility



Black-box

Tests behavior, not internal code



White-box

Tests internal logic & structure

By quality focus



Functional

Does it do the right thing?



Non-functional

Security, speed, usability, etc.

Key Takeaways

- ✓ A testing methodology is a strategy, not a single test — it guides what to test, when, and how.
- ✓ Good testing checks both correctness (verification) and real-world fit (validation).
- ✓ Testing operates at multiple levels: unit → integration → system → acceptance.
- ✓ It also spans multiple lenses: manual/automated, black-box/white-box, functional/non-functional.
- ✓ Specialized testing (security, performance) matters as much as functional correctness.